Ciclo 1 Universidad Don Bosco

Grupo: G01T

Materia:

Desarrollo de Software para Móviles

Nombre del trabajo:

Foro 1

Estudiantes:

Alan Anderson Vásquez Leiva #VL221407
Kelly Abigail Vázquez Rodríguez # VR220916
Guillermo Salvador Cartagena Mejia CM213015
Oliver Alejandro Erazo Reyes #ER231663
Julio César Posada Ramírez #PR222058

Docente:

Alexander Alberto Siguenza Campos

Universidad Don Bosco, 26 de abril de 2026

Índice

I. Introducción 3
II. Qué es Jetpack Compose y cuáles son sus ventajas 4
III. Manual de usuario 7
IV. Conclusión 14
V. Referencias 15

I. Introducción

El desarrollo de aplicaciones móviles ha evolucionado constantemente con el objetivo de crear soluciones más eficientes, rápidas y fáciles de mantener. En el ecosistema Android, durante muchos años la construcción de interfaces de usuario se basó en el uso de archivos XML junto con código en Java o Kotlin, separando la lógica de la presentación. Aunque este enfoque es funcional, puede volverse complejo en aplicaciones grandes debido a la cantidad de archivos y la gestión manual de las vistas.Con la llegada de nuevas herramientas y enfoques modernos, Google introdujo Jetpack Compose como una alternativa más simple y potente para el diseño de interfaces. Este nuevo modelo adopta un paradigma declarativo que permite construir la UI directamente en Kotlin, reduciendo la complejidad del desarrollo y mejorando la forma en que las aplicaciones reaccionan a los cambios de datos.En este documento se presenta una visión general de Jetpack Compose, sus principales ventajas, así como una descripción del flujo implementado en la aplicación desarrollada y una reflexión final sobre su uso en el desarrollo móvil actual.

II. Qué es Jetpack Compose y cuáles son sus ventajas

Jetpack Compose es el kit de herramientas moderno de Android desarrollado por Google para crear interfaces de usuario nativas. A diferencia del enfoque tradicional basado en archivos XML, Compose utiliza un paradigma declarativo, lo que significa que la interfaz se construye directamente con funciones en Kotlin describiendo cómo debe verse según el estado de los datos.

Este enfoque permite que la interfaz se actualice automáticamente cuando la información cambia, proceso conocido como recomposición, evitando la manipulación manual de elementos visuales como se hacía anteriormente.

Entre sus principales ventajas destacan:

Menos código: reduce la necesidad de múltiples archivos y simplifica el desarrollo.

Mayor claridad: todo se trabaja en Kotlin, facilitando la lectura y mantenimiento.

Interfaz reactiva: los cambios en los datos se reflejan automáticamente en la UI.

Desarrollo más rápido: permite visualizar cambios en tiempo real con herramientas de Android Studio.

Alta personalización: facilita la creación de diseños modernos, animaciones y uso de Material Design.

Interoperabilidad: puede integrarse con proyectos que aún utilizan XML, permitiendo una migración progresiva.

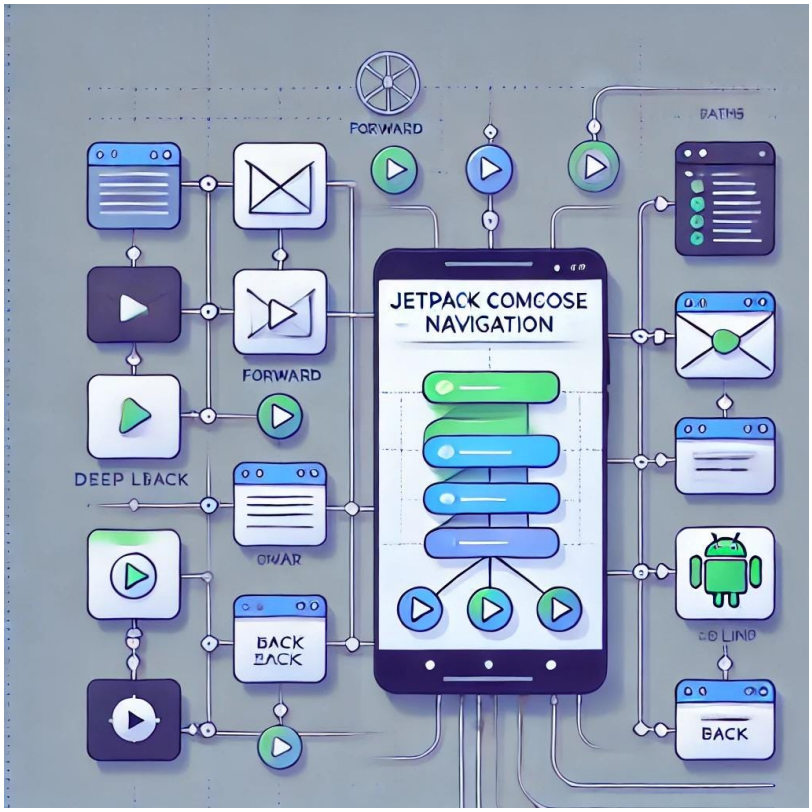
Descripción del flujo implementado y decisiones de diseño

El flujo de la aplicación inicia en una pantalla principal donde se muestra un listado de elementos obtenidos desde una fuente de datos. El usuario puede desplazarse por la lista y seleccionar un elemento, lo que lo dirige a una pantalla de detalle donde se presenta información más completa. Posteriormente, puede regresar a la pantalla anterior mediante la navegación integrada.

En cuanto a las decisiones de diseño, se optó por utilizar una arquitectura basada en ViewModel, lo que permite separar la lógica de negocio de la interfaz. Además, se aplicó un flujo de datos unidireccional, donde la información fluye desde el ViewModel hacia la interfaz, y las acciones del usuario regresan para actualizar el estado.

Se utilizó el componente **LazyColumn**, el cual reemplaza al RecyclerView y mejora el rendimiento al cargar únicamente los elementos visibles en pantalla. También se implementó navegación mediante Navigation Compose, facilitando la gestión entre pantallas dentro de una

sola actividad.Se utilizaron componentes de Material Design como Scaffold y TopAppBar para mantener una interfaz consistente, moderna y fácil de usar.



Conclusiones personales sobre el uso de Jetpack Compose

El uso de Jetpack Compose representa una mejora significativa en el desarrollo de aplicaciones Android, ya que simplifica la creación de interfaces y reduce la complejidad del código. La posibilidad de trabajar únicamente con Kotlin hace que el desarrollo sea más fluido y organizado.Aunque al inicio puede ser un poco difícil adaptarse al nuevo enfoque declarativo y al manejo del estado, con la práctica se vuelve más intuitivo. Además, permite desarrollar interfaces dinámicas de forma más rápida y con menos errores.Jetpack Compose es una herramienta moderna y eficiente que facilita el desarrollo de aplicaciones actuales, mejorando tanto la productividad del desarrollador como la calidad del producto final

III. Manual de usuario

Pantalla de Login. los usuarios pueden ingresar con su correo y contraseña.

Si la ingresamos mal nos aparecerá el siguiente mensaje.

Inicio de sesión
Correo electrónico
alan
Contraseña
Correo electrónico inválido
Iniciar sesión

Si lo ingresamos bien nos aparecerá un mensaje de bienvenida.

Nos aparecerá unas casillas que debemos de llenar con nuestras notas para que nos de nuestro promedio.

En caso que nuestro promedio sea bajo nos aparecerá el siguiente mensaje y nos dara oportunidad de ingresar otras notas o cerrar sesión .

En caso que aprobamos nos aparecerá el siguiente mensaje que hemos aprobado.

IV. Conclusión

El uso de Jetpack Compose representa un cambio importante en la forma de desarrollar aplicaciones Android, dejando atrás el enfoque tradicional basado en XML y adoptando un modelo más moderno, dinámico y eficiente. A lo largo del desarrollo, se puede notar una mejora en la organización del código, así como una mayor facilidad para construir y mantener interfaces de usuario.Aunque al inicio puede resultar un poco desafiante adaptarse al paradigma declarativo y al manejo del estado, con la práctica se convierte en una herramienta bastante intuitiva.Su integración con Kotlin y su capacidad para actualizar automáticamente la interfaz hacen que el desarrollo sea más ágil y menos propenso a errores.Jetpack Compose no solo simplifica el proceso de creación de interfaces, sino que también permite desarrollar aplicaciones más modernas, escalables y fáciles de mantener, por lo que su uso resulta altamente recomendable en proyectos actuales.

V. Referencias

Android Developers. (s. f.). *Get started with Jetpack Compose*. Android Developers. https://developer.android.com/develop/ui/compose/documentation

Android Developers. (s. f.). *Navigation with Compose*. Android Developers. https://developer.android.com/develop/ui/compose/navigation

Kotlin. (s. f.). *Kotlin for Android*. Kotlin Documentation. https://kotlinlang.org/docs/android-overview.html

Kotlin. (s. f.). *Basic syntax overview*. Kotlin Documentation. https://kotlinlang.org/docs/basic-syntax.html